

Teoría

1. ¿Cuales de los siguientes conceptos de RDBMS existen en MongoDB? En caso de no existir, ¿hay alguna alternativa? ¿Cual es?
 - Base de Datos
 - Tabla / Relacion
 - Fila / Tupla
 - Columna
 - Para Mongo no hay diferencia en lo que llamamos Base de Datos.
 - Lo que son las Tablas en las relacionales, en Mongo son Colecciones.
 - Las Relaciones en las relacionales, son documentos embebidos o referenciados desde otro documento.
 - Las Filas en las relacionales, son Documentos en Mongo.
 - Las Columnas en las relacionales, con Campos de un Documento en Mongo.
2. ¿Existen claves foraneas en MongoDB? ¿Qué diferencias existen con las bases de datos de tipo relacional?

Mongo **NO POSEE CLAVES FORÁNEAS**, se puede hacer una "**equivalencia**" ya que cada Documento en Mongo posee un `_id`, este id se puede usar para mediante la estrategia de Referencia generar relaciones entre Documentos. La diferencia de Mongo con las Relaciones es que **no mantiene la integridad referencial** con estos ids, si por ejemplo se borrara un Documento que es referenciado en otro Documento, uno como programador se tiene que encargar de ajustar la integridad referencial del id que quedo sin un Documento.

3. Para acelerar las consultas, MongoDB tiene soporte para índices. ¿Qué tipos de índices soporta?

MongoDB soporta índices de **campo único**, **compuestos**, **multikey** (para arreglos), **geoespaciales**, **de texto**, **hashed**, **wildcard** y **TTL**. Estos permiten acelerar las consultas según el tipo de datos y el patrón de acceso, optimizando búsquedas por igualdad, rangos, texto completo, ubicación geográfica, estructuras dinámicas o eliminando automáticamente documentos expirados.

- Índice simple: se crea sobre un solo campo.
- Índice compuesto: se crea sobre varios campos.
- Índice multikey: se utiliza para campos que contienen arrays.

- Índice de texto: permite realizar búsquedas de texto.
- Índice geoespacial: sirve para consultas relacionadas con ubicaciones y coordenadas.
- Índice hashed: almacena hashes de los valores de un campo, útil para sharding.
- Índice TTL (Time To Live): elimina automáticamente documentos después de un tiempo determinado.
- Índice único (unique): evita valores duplicados en un campo.
- Índice sparse: indexa solo documentos que contienen el campo especificado.

4. En MongoDB existen dos tipos de vistas. Explicar brevemente cuales son y que diferencias existen entre ellas. Además, mencionar algunos casos donde podría utilizarlas.

Vista Estándar: Es la Vista equivalente a las de SQL, se tiene almacenada la consulta y se ejecuta cada vez que necesitemos consultar los resultados que genera la Vista. Este tipo de Vista no ocupa espacio en Disco y no almacena los datos que genera.

Vista Materializada: Es una Vista que se ejecuta y se almacena su resultado en Disco, esto es para poder consultar los resultados de esa consulta de una forma rápida, la contra de esto es que en un período de tiempo, los resultados podrían llegar a cambiar y la información que mostramos puede quedar desactualizada.

5. Los documentos de una colección pueden diferir en la cantidad y tipos de campos. ¿Existen algunas formas de validar los elementos a insertar en una colección para evitar esta disparidad?

Aunque Mongo es Schema less puede proveer mecanismos para validar esos elementos, pero las validaciones principales se deben hacer en la aplicación.

6. **MongoDB tiene soporte para transacciones, pero no es igual que el de los RDBMS. ¿Cuál es el alcance de una transacción en MongoDB?**

La principal diferencia es que en MongoDB las operaciones son atómicas a nivel de documento. Si se necesita una transacción, esta puede abarcar múltiples documentos y colecciones, aunque su uso no es el mecanismo habitual. En los RDBMS, las transacciones son el mecanismo principal para garantizar la consistencia entre múltiples filas y tablas.

7. Las relaciones entre documentos en MongoDB pueden establecerse mediante documentos embebidos o referencias. Investigar como se implementa cada una y analizar las ventajas y desventajas de cada una, comparándola con la forma estándar de establecer relaciones en una base de datos relacional.

- **Embebidos:** Dentro de un documento tenemos almacenados todos los Documentos con los que se relaciona.
 - **Ventajas:**

- Cuando haces una consulta por un documento que tiene otros embebidos, eso hace que recuperemos toda la información de una lectura. Esta recuperación es rápida.
 - **Desventaja:**
 - Los documentos soportan máximo 16 mb, lo que puede hacer que no entre toda la información embebida.
 - Tener documentos embebidos puede generar consultas más complejas para poder acceder a esa información.
 - Los documentos embebidos en relaciones muchos a muchos generan redundancia de la información.
 - **Referencia:** Consiste en almacenar un campo con el id del otro Documento al que se referencia.
 - **Ventajas:**
 - Las referencias pueden crecer y es muy difícil tener problemas de memoria en los documentos por esta razón en específico.
 - Facilita actualizar datos referenciados.
 - Evita duplicar información.
 - **Desventaja:**
 - Complejiza las consultas cuando se quiere acceder a un Documento y a todos con los que se relaciona de esta forma.
 - El mantenimiento de la integridad referencial recae en el programador.
7. Tomando como referencia el modelo de los trabajos prácticos anteriores y suponiendo que este podría mapearse a una base de datos en MongoDB, proponer algunos casos donde la relación sería conveniente mapearla como referencia y otros como documentos embebidos. Justificar la elección.

Consultas

1. Crear una nueva base de datos llamada "tours" y una colección llamada "recorridos".

```
use tours // use para usar una db
db.createCollection("recorridos") // createCollection para crear
colecciones xd
```

2. En la nueva colección, utilizando el comando correspondiente, insertar el siguiente documento:

```
db.recorridos.insertOne({
  contenido
})
```

3. Recuperar la información insertada usando `db.recorridos.find()` (puede agregarse `.pretty()` al final para ver los datos indentados). ¿Qué diferencia se observa entre el documento insertado y el documento recuperado?

La diferencia que se observa es que cuando haces el find, te aparece el documento con el `_id` puesto.

3. Agregar a la colección, utilizando un solo comando, los documentos especificados en el archivo "material_adicional_1.json" adjunto a esta practica.

```
db.recorridos.insertMany([
  {}, {}, {}, ...
])
```

4. Sobre la coleccion cargada en el punto anterior, realizar las siguientes operaciones:
 - Actualizar el recorrido "Cultural Odyssey" para que su total de kilometros sea 12.
 - Actualizar el listado de stops del recorrido "Delta Tour" para agregar "Tigre".
 - Aumentar un 10% el precio de todos los recorridos.
 - Eliminar el recorrido con nombre "Temporal Route".
 - Crear el array de etiquetas (tags) para la ruta "Urban Exploration" y agregar el elemento "Gastronomia" a dicho arreglo.

```
db.recorridos.updateOne(
  { nombre: "Cultural Odyssey" },
  { $set: { totalKm: 12 } }
)
```

```
db.recorridos.updateOne(
  { nombre: "Delta Tour" },
  { $push: { stops: "Tigre" } }
)
```

```
db.recorridos.updateMany(
  { },
  { $mul: { precio: 1.1 } }
)
```

```
db.recorridos.deleteOne(  
  { nombre: "Temporal Route" }  
)
```

```
db.recorridos.updateOne(  
  { nombre: "Urban Exploration" },  
  { $push: { tags: "Gastronomia" } } // push crea el array si no existe  
)
```

5. Sobre la coleccion generada anteriormente, realizar las siguientes consultas utilizando la funcion find():

- Obtener la ruta con nombre "Museum Tour".
- Las rutas con precio superior a \$60.000.
- Las rutas con precio superior a \$50.000 y con un total de kilometros mayor a 10.
- Las rutas que incluyan el stop "San Telmo".
- Las rutas que incluyan el stop "Recoleta" y no el stop "Plaza Italia".
- El nombre y el total de km (si es que posee) de las rutas que incluyan el stop "Delta" y tengan un precio menor a \$50.000.
- Las rutas que incluyen tanto "San Telmo" como "Recoleta" y "Avenida de Mayo" entre sus stops.
- Solo el nombre de las rutas que dispongan de mas de 5 stops.
- Las rutas que no tengan definido el total de sus kilometros.
- Los nombres y el listado de stops de aquellas rutas que incluyen algun museo en sus recorridos.
- La cantidad total de elementos que posee la coleccion.

```
db.recorridos.find(  
  { nombre: "Museum Tour" }  
)
```

```
db.recorridos.find(  
  { precio: { $gt: 60000 } }  
)
```

```
db.recorridos.find(  
  {  
    precio: { $gt: 50000 },  
    kilometros: { $gt: 10 }  
  })
```

```
}  
)
```

```
db.recorridos.find(  
  { stops: "San Telmo" }  
)
```

```
db.recorridos.find(  
  {  
    $and: [  
      { stops: "Recoleta" },  
      { stops: { $ne: "Plaza Italia" } }  
    ]  
  }  
)
```

```
db.recorridos.find(  
  {  
    stops: {  
      $all: ["Recoleta"],  
      $nin: ["Plaza Italia"]  
    }  
  }  
)
```

```
db.recorridos.find(  
  {  
    stops: "Delta",  
    precio: { $lt: 50000 }  
  },  
  {  
    nombre: 1,  
    kilometros: 1,  
    _id: 0  
  }  
)
```

```
db.recorridos.find(  
  {  
    stops: { $all: ["San Telmo", "Recoleta", "Avenida de Mayo"] }  
  },  
)
```

```

db.recorridos.find(
  {
    "stops.5": { $exists: true } // estamos preguntando si existe la
pos 6
  },
  {
    nombre: 1,
    _id: 0
  }
)

db.recorridos.find(
  {
    $expr: {
      $gt: [
        { $size: "$stops" },
        5
      ]
    },
    {
      nombre: 1,
      _id: 0
    }
  }
)

```

```

db.recorridos.find(
  {
    kilometros: { $exists: false }
  }
)

```

```

db.recorridos.find(
  {
    stops: {
      $regex: "Museo"
    }
  },
  {
    nombre: 1,
    stops: 1,
    _id: 0
  }
)

```

```
}  
)
```

```
db.recorridos.countDocuments() // cantidad de documentos de la colección
```

Aggregation Framework 🦊

El **Aggregation Framework** de MongoDB es una herramienta sumamente poderosa diseñada para realizar operaciones de **procesamiento, transformación y análisis de datos** directamente sobre las colecciones. A diferencia de las consultas simples, este framework permite manipular grandes volúmenes de datos para obtener información valiosa en tiempo real, generar reportes y ejecutar cálculos complejos a escala.

¿Qué es y cómo funciona el Pipeline de Agregación?

El concepto central es el **pipeline (tubería) de agregación**, el cual consiste en una serie de **etapas (stages)** por las que pasan los documentos.

- **Flujo de ejecución:** Una vez iniciado, el optimizador de consultas de MongoDB organiza el proceso y crea un cursor para manejar el flujo de documentos.
- **Procesamiento por etapas:** Cada etapa realiza una operación específica (como filtrar, agrupar o calcular valores) y pasa los documentos resultantes a la siguiente etapa hasta llegar al resultado final.

Principales Etapas y sus Usos

1. `$match` (Filtrado)

- **Función:** Filtra los documentos para que solo aquellos que cumplan con criterios específicos pasen a la siguiente etapa.
- **Cuándo usarlo:** Debe colocarse **al principio del pipeline** para reducir la cantidad de datos procesados y aprovechar los índices existentes, lo que optimiza significativamente el rendimiento.

2. `$unwind` (Descomposición de arreglos)

- **Función:** "Descompone" o aplanar un campo que contiene un arreglo, generando un documento de salida por cada elemento del array.
- **Cuándo usarlo:** Es crucial cuando se necesita realizar operaciones sobre elementos individuales de un arreglo, como agrupar libros por género cuando estos están guardados

en una lista dentro de cada venta.

3. `$group` (Agrupación y Cálculo)

- **Función:** Combina documentos en grupos basados en una **clave de grupo (group key)**.
- **Para qué sirve:** Permite realizar cálculos como sumas (`$sum`), promedios (`$avg`) o conteos sobre los datos agrupados. Por ejemplo, se usa para calcular los ingresos totales por cada género de libro.

4. `$project` (Modelado de datos)

- **Función:** Resuena o "limpia" los documentos finales.
- **Capacidades:** Permite incluir o excluir campos específicos, renombrar campos existentes o crear campos nuevos para que el resultado sea más legible para los interesados. Se recomienda usarlo **hacia el final** del pipeline para no perder campos necesarios en etapas previas de optimización.

5. Otras Etapas Relevantes:

- `$sort` : Ordena los documentos (ascendente o descendente).
- `$limit` : Restringe el número de documentos entregados.
- `$lookup` : Realiza una operación similar a un "left outer join" de SQL para traer datos relacionados de otra colección.

Mejores Prácticas y Secuenciación

El orden de las etapas es vital porque impacta tanto en la **precisión de los resultados** como en el **rendimiento** del sistema.

- **Filtrar temprano:** Siempre use `$match` al inicio para disminuir el tamaño del conjunto de datos.
- **Optimización automática:** MongoDB incluye un optimizador que puede combinar o reordenar etapas automáticamente para mejorar la eficiencia sin alterar el resultado final.
- **Visualización:** Herramientas como **MongoDB Compass** permiten visualizar cómo se transforman los documentos en cada etapa del pipeline.

¿Cuándo usar Aggregation vs. Find?

- **Operaciones simples:** Para la recuperación básica de documentos, es preferible utilizar operaciones de búsqueda estándar (`find`).
- **Transformaciones complejas:** El Aggregation Framework es la opción ideal cuando el caso de uso requiere múltiples pasos de transformación, reestructuración de datos o cálculos analíticos.

Consultas

1. Obtener una muestra de 5 rutas aleatorias de la coleccion.

```
db.recorridos.aggregate([
  {
    $sample : {
      size : 5
    }
  }
])
```

2. Extender la consulta anterior para incluir en el resultado toda la informacion de cada una de las stops. Tener en cuenta que pueden ligarse por su codigo.

```
db.recorridos.aggregate([
  {
    $sample : {
      size : 5
    }
  },
  {
    $lookup: {
      from: "stop",
      localField: "stops",
      foreignField: "codigo",
      as: "stops_info"
    }
  }
])
```

3. Obtener la informacion de las rutas (incluyendo la de sus stops) que tengan un precio mayor o igual a \$90.000.

```
db.recorridos.aggregate([
  {
    $match : {
      precio : {
        $gte : 90000
      }
    }
  },
  {
    $lookup: {
```

```

        from: "stop",
        localField: "stops",
        foreignField: "codigo",
        as: "stops_info"
    }
}
])

```

4. Obtener la informacion de las rutas que tengan 5 stops o mas.

```

db.recorridos.aggregate([
    {
        $match: {
            "stops.4": { $exists: true }
        }
    },
    {
        $project: {
            nombre: 1
            _id: 0
        }
    }
])

db.recorridos.aggregate([
    {
        $match: {
            $expr: {
                $gte: [{ $size: "$stops" }, 5]
            }
        }
    },
    {
        $project: {
            nombre: 1,
            _id: 0
        }
    }
])

```

5. Obtener la informacion de las rutas que tengan incluido en su nombre el string "111".

```

db.recorridos.aggregate([
    {

```

```

        $match : {
            nombre : {
                $regex : "111"
            }
        }
    }
}
])

```

6. Obtener solo las stops de la ruta con nombre "Route100".

```

db.recorridos.aggregate([
    {
        $match : {
            nombre : "Route100"
        }
    },
    {
        $lookup: {
            from: "stop",
            localField: "stops",
            foreignField: "codigo",
            as: "stops_info"
        }
    },
    {
        $project : {
            stops_info: 1
            _id: 0
        }
    }
])

```

7. Obtener la información del stop que mas apariciones tiene en rutas.

```

db.recorridos.aggregate([
    {
        $unwind : "$stops"
    },
    {
        $group : {
            _id : "$stops",
            apariciones : {
                $sum : 1
            }
        }
    }
])

```

```

    }
  },
  {
    $sort : {
      apariciones : -1
    }
  },
  {
    $limit : 1
  },
  {
    $lookup : {
      from : "stop",
      localField : "_id",
      foreignField : "codigo",
      as : "stops_info"
    }
  }
}
])

```

\$unwind → { stops: "A", ... }
 \$group → { _id: "A", apariciones: 2 } ← "stops" desaparece, pasa a ser "_id"
 \$sort → el de más apariciones primero
 \$limit → { _id: "A", apariciones: 2 }
 \$lookup → una por _id:"A" con codigo:"A" de la colección stop

8. Obtener las rutas con precio inferior a \$15.000. Agregar a cada una una nueva propiedad que especifique la cantidad de stops que posee. Crear una nueva colección llamada "rutas_economicas" y almacenar estos elementos.

```

db.recorridos.aggregate([
  {
    $match : {
      precio : {
        $lt : 15000
      }
    }
  },
  {
    $addFields : {
      cantidadStops : {
        $size : "$stops"
      }
    }
  }
])

```

```

    },
    {
        $out : "rutas_economicas"
    }
  ]
})

```

9. Por cada stop existente en la coleccion, calcular el precio promedio de las rutas que la incluyen.

```

db.recorridos.aggregate([
  {
    $unwind : "$stops"
  },
  {
    $group : {
      _id: "$stops",
      precioPromedio : {
        $avg : "$precio"
      }
    }
  }
])

```

`$unwind` → un documento por cada stop de cada ruta

```

{ ruta: "Route100", stops: "A", precio: 100 }
{ ruta: "Route100", stops: "B", precio: 100 }
{ ruta: "Route200", stops: "A", precio: 200 }

```

`$group` → agrupa por stop y promedia el precio

```

{ _id: "A", precioPromedio: 150 } ← (100 + 200) / 2
{ _id: "B", precioPromedio: 100 }

```

Ejercicios de los Parciales

MongoDB

8. Suponiendo dos colecciones: una llamada "autores", donde cada elemento representa a un autor identificado por su `_id`, y otra llamada "documentos", en la que cada elemento referencia a sus autores mediante un arreglo "autorIds", resolver con Aggregation Framework: por cada autor, calcular la cantidad de documentos en los que participa y el año de publicación promedio de esos documentos; devolver únicamente los autores con más de 10 documentos, ordenados por cantidad de documentos de forma descendente.

```

db.documentos.aggregate([
  {
    $unwind : "$autorIds"
  },
  {
    $group : {
      _id : "$autorIds",
      cantDocumentos : {
        $sum : 1
      },
      añoPromedioPublicacion : {
        $avg : "$añoPublicacion"
      }
    }
  },
  {
    $match : {
      cantDocumentos : {
        $gt : 10
      }
    }
  },
  {
    $sort : {
      cantDocumentos : -1
    }
  }
])

```

\$unwind → un doc por cada autor dentro de cada documento
 \$group → agrupa por autor, cuenta docs y promedia año
 \$match → filtra los que tienen más de 10 ← DESPUÉS del group, porque cantDocumentos no existe antes
 \$sort → ordena descendente por cantidad

MongoDB

- Suponiendo una colección "cursos" en la que cada documento referencia a su instructor mediante un campo "instructorId", resolver con Aggregation Framework: por cada instructor, calcular la cantidad de cursos que dicta y el precio promedio de esos cursos; devolver únicamente los instructores con valoración mayor a 8, ordenados por precio promedio de forma descendente.

```

db.cursos.aggregate([
  {
    $lookup : {
      from : "instructores",
      localField : "instructorId",
      foreignField : "_id",
      as : "instructor"
    }
  },
  {
    $unwind: "$instructor"
  },
  {
    $match : {
      "instructor.valoracion" : {
        $gt : 8
      }
    }
  },
  {
    $group : {
      _id : "$instructorId",
      cantCursos : {
        $sum : 1
      },
      precioPromedio : {
        $avg : "$precio"
      }
    }
  },
  {
    $sort : {
      precioPromedio : -1
    }
  }
])

```

\$lookup → instructor: [{ valoracion: 9, ... }] ← array
 \$unwind → instructor: { valoracion: 9, ... } ← objeto, ahora sí podés acceder con dot notation
 \$match → filtra valoracion > 8
 \$group → cuenta cursos y promedia precio por instructor
 \$sort → ordena por precio promedio descendente

Suponga que el modelo de la plataforma de recetas se persiste en MongoDB con las siguientes dos colecciones:

- **ingredientes**: {_id: ObjectId, nombre: String, unidadMedida: String, caloriasXUnidad: int}
- **recetas**: {_id: ObjectId, titulo: String, tiempoPreparacion: int, dificultad: String, fecha: Date, ingredientIds: [ObjectId, ObjectId, ...]}

Utilizando el Aggregation Framework de MongoDB, construya un pipeline que realice las siguientes operaciones en ese orden:

1. Para cada ingrediente, calcular: la cantidad de recetas en las que aparece y el tiempo de preparación promedio de esas recetas.
2. Incluir en el resultado solo los ingredientes que aparecen en más de 3 recetas.
3. Ordenar los resultados por cantidad de recetas de forma descendente.
4. Mostrar únicamente: el nombre del ingrediente, la cantidad de recetas y el tiempo promedio.

```
db.recetas.aggregate([
  {
    $unwind : "$ingredienteIds"
  },
  {
    $group : {
      _id : "$ingredienteIds",
      cantRecetas : {
        $sum : 1
      },
      tiempoPrepProm : {
        $avg : "$tiempoPreparacion"
      }
    }
  },
  {
    $match : {
      cantRecetas : {
        $gt : 3
      }
    }
  },
  {
    $sort : {
      cantRecetas : -1
    }
  },
])
```

```

{
  $lookup : {
    from : "ingredientes",
    localField : "_id",
    foreignField : "_id",
    as : "ingrediente"
  }
},
{
  $unwind : "$ingrediente"
},
{
  $project : {
    "ingrediente.nombre" : 1,
    cantRecetas : 1,
    tiempoPrepProm : 1
  }
}
])

```

Ejercicios de Claudio

Find

1. Tenés una colección "empleados" donde cada documento tiene: nombre, sector, salario, habilidades (array) y activo (boolean). Obtener el nombre y salario de los empleados que trabajen en el sector "IT", tengan la habilidad "Python" y ganen más de \$200.000. Excluir el _id.

```

db.empleados.find(
{
  sector : "IT",
  habilidades : "Python",
  salario : {
    $gt : 200000
  }
},
{
  nombre : 1,
  salario : 1
  _id : 0
}
)

```

```
}  
)
```

2. De la misma colección, obtener los empleados que tengan **al menos 4 habilidades** y estén **activos**. Mostrar solo nombre y habilidades.

```
db.empleados.find(  
  {  
    activo : true,  
    $expr : {  
      $gte : [  
        {  
          $size : "$habilidades"  
        },  
        4  
      ]  
    }  
  },  
  {  
    nombre : 1,  
    habilidades : 1  
    _id : 0  
  }  
)
```

3. Obtener los empleados que tengan la habilidad "Java" pero **no** tengan "Python". Mostrar nombre y sector.

```
db.empleados.find(  
  {  
    $and : [  
      {  
        habilidades : "Java"  
      },  
      {  
        habilidades : {  
          $ne : "Python"  
        }  
      }  
    ]  
  },  
  {  
    nombre : 1,  
    sector : 1  
  }  
)
```

```
    _id : 0
  }
)
```

4. Obtener los empleados cuyo nombre contenga la palabra "Lopez" (sin importar mayúsculas). Mostrar solo el nombre.

```
db.empleados.find(
  {
    nombre : {
      $regex : "Lopez",
      $options : "i"
    }
  },
  {
    nombre : 1,
    _id : 0
  }
)
```

Aggregation

5. Dada la colección "ventas" donde cada documento tiene: vendedor, producto, categoria, monto y fecha. Obtener por cada vendedor el monto total vendido y el promedio por venta. Mostrar solo los vendedores que hayan superado \$500.000 en total, ordenados de mayor a menor.

```
db.ventas.aggregate([
  {
    $group : {
      _id : "$vendedor",
      montoTotal : {
        $sum : "$monto"
      },
      promedioVenta : {
        $avg : "$monto"
      }
    }
  },
  {
    $match : {
      montoTotal : {
        $gt : 500000
      }
    }
  }
])
```

```

    }
  },
  {
    $sort : {
      montoTotal : -1
    }
  }
]
])

```

6. Sobre la misma colección: obtener las 3 categorías con mayor monto total vendido. Mostrar la categoría y el total.

```

db.ventas.aggregate([
  {
    $group : {
      _id : "$categoria",
      montoTotal : {
        $sum : "$monto"
      }
    }
  },
  {
    $sort : {
      montoTotal : -1
    }
  },
  {
    $limit : 3
  },
  {
    $project : {
      _id : 0,
      categoria : "$_id",
      montoTotal : 1
    }
  }
]
])

```

7. Dada la colección "pedidos" donde cada documento tiene: cliente, estado ("pendiente", "enviado", "entregado"), productos (array de strings) y total. Obtener por cada estado la cantidad de pedidos y el total promedio. Ordenar por cantidad de pedidos descendente.

```

db.pedidos.aggregate([
  {
    $group : {
      _id : "$estado",
      cantPedidos : {
        $sum : 1
      },
      totalPromedio : {
        $avg : "$total"
      }
    }
  },
  {
    $sort : {
      cantPedidos : -1
    }
  }
])

```

8. Dada la colección "facultad" donde cada documento representa un alumno con: nombre , carrera , materias (array de strings) y promedio . Obtener por cada carrera:

- La cantidad de alumnos
- El promedio general de la carrera
- Mostrar solo las carreras con **más de 10 alumnos** y promedio **mayor a 7**

```

db.facultad.aggregate([
  {
    $group : {
      _id : "$carrera",
      cantAlumnos : {
        $sum : 1
      },
      promedioGeneral : {
        $avg : "$promedio"
      }
    }
  },
  {
    $match : {
      cantAlumnos : {
        $gt : 10
      },
      promedioGeneral : {

```

```

    }
  }
}
])

```

Combinados con \$lookup (nivel parcial real)

9. Tenés dos colecciones:

- "productos" : cada producto tiene nombre, precio, categoriaId
- "categorias" : cada categoría tiene _id, nombre, descripcion

Obtener para cada producto su información completa incluyendo el nombre de la categoría.

Mostrar solo los productos con precio mayor a \$10.000, ordenados por precio descendente.

```

db.productos.aggregate([
  {
    $match : {
      precio : {
        $gt : 10000
      }
    }
  },
  {
    $lookup : {
      from : "categorias",
      localField : "categoriaId",
      foreignField : "_id",
      as : "categoria"
    }
  },
  {
    $unwind : "$categoria"
  },
  {
    $sort : {
      precio : -1
    }
  }
])

```

10. Tenés dos colecciones:

- "posts" : cada post tiene titulo, autorId, tags (array), likes

- "usuarios": cada usuario tiene `_id`, `nombre`, `pais`

Obtener el **post con más likes** de autores del país "Argentina", incluyendo el nombre del autor. Mostrar título, likes y nombre del autor.

```
db.posts.aggregate([
  {
    $lookup : {
      from : "usuarios",
      localField : "autorId",
      foreignField : "_id",
      as : "autor"
    }
  },
  {
    $unwind : "$autor"
  },
  {
    $match : {
      "autor.pais" : "Argentina"
    }
  },
  {
    $sort : {
      likes : -1
    }
  },
  {
    $limit : 1
  },
  {
    $project :{
      titulo : 1,
      likes : 1,
      "autor.nombre" : 1,
      _id : 0
    }
  }
])
```

11. Tenés las colecciones "facturas" y "clientes". Cada factura tiene: `clienteId`, `items` (array de strings), `total` y `pagada` (boolean). Obtener por cada cliente el monto total facturado considerando **solo las facturas pagas**, junto con el nombre del cliente. Devolver únicamente los clientes con más de \$100.000 facturado, ordenados descendente.


```

db.facturas.aggregate([
  {
    $match : {
      pagada : true
    }
  },
  {
    $lookup : {
      from : "clientes",
      localField : "clienteId",
      foreignField : "_id",
      as : "cliente"
    }
  },
  {
    $unwind : "$cliente"
  },
  {
    $group : {
      _id : "$cliente._id",
      totalFacturado : {
        $sum : "$total"
      },
      nombre: {
        $first: "$cliente.nombre"
      }
    }
  },
  {
    $match : {
      totalFacturado : {
        $gt : 100000
      }
    }
  },
  {
    $sort : {
      totalFacturado : -1
    }
  }
])

```